

OPTIMIZATION AND REGULARIZATION TECHNIQUES

Complete Study Notes • Deep Learning Unit

1. OPTIMIZATION FOR TRAINING DEEP MODELS

| Gradient Descent Variants

Variant	Data Used	Update Frequency	Stability	Speed
Batch GD	Full dataset	Once per epoch	Very stable	Slow
Stochastic GD (SGD)	1 sample	Each sample	Noisy	Fast per update
Mini-Batch GD	Small batch (e.g. 32)	Each mini-batch	Balanced	Most efficient

| Mini-Batch Gradient Descent

- Splits data into small batches (typically 32, 64, 128 samples)
- **Faster and more stable than pure SGD**
- **Most commonly used in practice**

► Working / Flowchart

STEP 1: Forward Pass → Compute Predictions
 STEP 2: Compute Loss (e.g. Cross Entropy, MSE)
 STEP 3: Backward Pass → Compute Gradients
 STEP 4: Update Weights using optimizer rule
 STEP 5: Repeat for all mini-batches → 1 Epoch Done

► Numerical / Exam Focus

- Weight update calculation (see Section 2)
- Effect of batch size: larger batch → smoother gradient, more memory
- Iterations per epoch = Total samples ÷ Batch size

Iterations per epoch = $N / \text{batch_size}$ (e.g. 1000 samples, batch=32 → ~31 iterations)

2. GRADIENT DESCENT UPDATE RULE

Basic Update Equation

$$\theta = \theta - \eta \cdot \nabla J(\theta)$$

- θ (theta)
- η (eta) = Learning Rate
- $\nabla J(\theta)$ = Gradient of Loss w.r.t. parameters

► Worked Example

Given: $\theta = 2.0$, $\eta = 0.1$, $\nabla J(\theta) = 3.0$ Update: $\theta_{\text{new}} = 2.0 - (0.1 \times 3.0) = 2.0 - 0.3 = 1.7 \rightarrow$
Weight updated to 1.7

► Effect of Learning Rate

Learning Rate	Effect	Problem
Too HIGH (e.g. 1.0)	Overshoots minimum	Divergence / oscillation
Too LOW (e.g. 0.0001)	Slow convergence	Takes too many epochs
Just Right	Smooth convergence	Optimal performance

3. ADVANCED OPTIMIZATION TECHNIQUES

1. Fixed Learning Rate

- A constant learning rate is maintained throughout training.
- Simple to implement and commonly used in basic models.
- Its limitation is that it lacks the ability to adapt on different training phases which may create sub optimal results.

2. Learning Rate Schedules

These techniques reduce the learning rate over time based on predefined rules to improve convergence:

- **Step Decay:** Reduces the learning rate by a fixed factor at set intervals (every few epochs).
- **Exponential Decay:** Continuously decreases the learning rate exponentially over training time.
- **Polynomial Decay:** Learning rate decays polynomially, offering smoother transitions compared to step or exponential methods.

3. Adaptive Learning Rate Methods

Adaptive methods adjust the learning rate dynamically based on gradient information, allowing better updates per parameter:

- **AdaGrad:** [AdaGrad](#) adapts the learning rate per parameter based on the squared gradients. It is effective for sparse data but may decay too quickly.
- **RMSprop:** [RMSprop](#) builds on AdaGrad by using a moving average of squared gradients to prevent aggressive decay.
- **Adam (Adaptive Moment Estimation):** [Adam](#) combines RMSprop with momentum to provide stable and fast convergence; widely used in practice.

4. Cyclic Learning Rate

- The learning rate oscillates between a minimum and maximum value in a cyclic manner throughout training.
- It increases and then decreases the learning rate linearly in each cycle.
- Benefits include better exploration of the loss surface and leading to faster convergence.

5. Decaying Learning Rate

- Gradually reduces the learning rate as training progresses.
- Helps the model take more precise steps towards the minimum. This improves stability in later epochs.

Momentum

- Uses past gradients to smooth updates (like a ball rolling down a hill)
- **Helps escape local minima and speeds up convergence**

$$v = \beta \cdot v + (1-\beta) \cdot \nabla J \quad \rightarrow \quad \theta = \theta - \eta \cdot v$$

RMSprop

- **Adapts learning rate per parameter**
- Uses exponentially weighted moving average of squared gradients
- Good for non-stationary objectives

Adam Optimizer ★ VERY IMPORTANT

Adam = Adaptive Moment Estimation = Momentum + RMSprop

Component	Formula	Purpose
1st Moment (Momentum)	$m = \beta_1 \cdot m + (1-\beta_1) \cdot \nabla J$	Mean of gradients
2nd Moment (RMSprop)	$v = \beta_2 \cdot v + (1-\beta_2) \cdot \nabla J^2$	Variance of gradients
Bias Correction	$\hat{m} = m / (1-\beta_1^t), \hat{v} = v / (1-\beta_2^t)$	Corrects early-step bias
Weight Update	$\theta = \theta - \eta \cdot \hat{m} / (\sqrt{\hat{v}} + \epsilon)$	Final parameter update

★ Default Adam values: $\beta_1=0.9$, $\beta_2=0.999$, $\epsilon=1e-8$

Comparison of Optimizers

Optimizer	Adapts LR?	Uses Momentum?	Best For
SGD	No	No	Simple problems, large datasets
SGD + Momentum	No	Yes	Faster convergence than SGD
RMSprop	Yes	No	Non-stationary, RNNs
Adam	Yes	Yes	Most deep learning tasks ✓

Optimizer	Main Formula	Meaning
Gradient Descent	$w = w - \eta \frac{\partial L}{\partial w}$	Basic weight update
Momentum	$v_t = \beta v_{t-1} + (1 - \beta) \frac{\partial L}{\partial w}, \quad w = w - \eta v_t$	Uses previous gradients (velocity)
AdaGrad	$G_t = G_{t-1} + \left(\frac{\partial L}{\partial w}\right)^2, \quad w = w - \frac{\eta}{\sqrt{G_t + \epsilon}} \frac{\partial L}{\partial w}$	Adaptive learning rate
RMSProp	$E[g^2]_t = \beta E[g^2]_{t-1} + (1 - \beta) g_t^2, \quad w = w - \frac{\eta}{\sqrt{E[g^2]_t + \epsilon}} g_t$	Moving average of squared gradients
Adam	$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t +$ $v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2 +$ $w = w - \eta \frac{m_t}{\sqrt{v_t + \epsilon}}$	Combines Momentum + RMSProp

Optimizer	Pros	Cons
Gradient Descent (SGD)	Simple, low memory usage, good generalization	Slow convergence, oscillations, sensitive to learning rate
Momentum	Faster convergence, reduces zig-zag movement, smoother updates	Can overshoot minima, needs momentum tuning
AdaGrad	Adaptive learning rates, works well for sparse data and NLP	Learning rate continuously decreases and may become too small
RMSProp	Fixes AdaGrad problem, stable learning, good for RNNs	Requires tuning of decay parameter and learning rate
Adam	Fast convergence, combines Momentum + RMSProp, works well for most deep learning tasks	Higher memory usage, sometimes poorer generalization than SGD

4. LOCAL AND GLOBAL OPTIMA

Type	Description	Gradient At Point
Global Minimum	Lowest point in entire loss landscape	0 (ideal)
Local Minimum	Lowest in a region but not globally	0 (trap)
Saddle Point	Flat region — not a minimum or maximum	~0 (problematic)

CHALLENGE: Deep neural networks have non-convex loss functions → many local minima and saddle points. Modern optimizers (especially Adam) help navigate these challenges. Note: Saddle points are MORE common than local minima in high-dimensional spaces.

5. LEARNING RATE STRATEGIES

Why Learning Rate Matters

- Controls step size during gradient descent
- Too large → divergence; Too small → slow training

Decay Strategies

Strategy	Formula / Idea	When to Use
Constant LR	$\eta = \text{fixed}$	Simple baseline
Step Decay	$\eta = \eta_0 \times \text{drop}^{\lfloor \text{epoch}/\text{step} \rfloor}$	Reduce every N epochs
Exponential Decay	$\eta = \eta_0 \times e^{(-k \cdot \text{epoch})}$	Smooth gradual decrease
Warm Restarts	LR rises then falls cyclically	Escape local minima

► Numerical Example – Exponential Decay

$$\eta_0 = 0.1, \quad k = 0.1, \quad \text{epoch} = 5 \rightarrow \eta = 0.1 \times e^{(-0.5)} \approx 0.0607$$

6. TRAIN / DEV / TEST SET CONFIGURATION

Set	Purpose	Used For	Typical Split
Training Set	Learn patterns from data	Weight updates	60–80%
Validation (Dev) Set	Tune hyperparameters	Model selection	10–20%
Test Set	Final evaluation ONLY	Unbiased performance	10–20%

★ **NEVER** use the test set during training or hyperparameter tuning — this causes data leakage!

► Best Practices

- Shuffle data before splitting to avoid ordering bias
- Use stratified splitting for imbalanced datasets
- Ensure test set is representative of real-world distribution
- **Dev set detects overfitting; test set measures generalization**

7. BIAS-VARIANCE TRADE-OFF

Concept	Definition	Symptom
High Bias	Model too simple, misses patterns	High training error
High Variance	Model too complex, memorizes noise	Low train error, high test error

Ideal Model	Right balance	Low train and test error
-------------	---------------	--------------------------

BIAS: Error from wrong assumptions (e.g. fitting a line to non-linear data)
 VARIANCE: Error from sensitivity to small fluctuations in training data
 Trade-off: Decreasing bias often increases variance and vice versa.
 Goal: Find the sweet spot using regularization, more data, or better architecture.

8. UNDERFITTING AND OVERFITTING

Problem	Training Error	Test Error	Cause	Solution
Underfitting	HIGH	HIGH	Model too simple	Increase complexity, more features
Overfitting	LOW	HIGH	Model memorizes data	Regularization, dropout, more data
Good Fit	LOW	LOW	Right complexity	Continue monitoring

UNDERFITTING → High Bias problem
 OVERFITTING → High Variance problem
 Exam Tip: On a graph, if training accuracy >> validation accuracy → overfitting. If both are low → underfitting.

9. REGULARIZATION TECHNIQUES

L1 Regularization (Lasso)

$$J = J_0 + \lambda \cdot \sum |w|$$

- Adds absolute value of weights as penalty
- Produces SPARSE weights (many weights become exactly 0)
- Useful for feature selection

L2 Regularization (Ridge / Weight Decay)

$$J = J_0 + \lambda \cdot \sum w^2$$

- Adds squared weights as penalty
- Produces SMOOTH weights (weights shrink but rarely reach 0)
- Most commonly used in deep learning

L1 vs L2 Comparison

Property	L1 (Lasso)	L2 (Ridge)
Penalty	$ w $	w^2
Weights	Sparse (many = 0)	Small but non-zero
Use Case	Feature selection	Prevent overfitting
Differentiable?	No (at 0)	Yes

Dropout

- **Randomly deactivates neurons during training (e.g. $p=0.5$ drops 50%)**
- Prevents co-adaptation of neurons
- Acts as ensemble method — different sub-networks each forward pass
- During inference: all neurons active, weights scaled by $(1-p)$

Example: If dropout rate = 0.3 $\rightarrow$ 30% of neurons are randomly set to 0 each training step. The model learns redundant representations $\rightarrow$ better generalization.

★ λ (lambda): Higher $\lambda \rightarrow$ stronger regularization $\rightarrow$ simpler model. Too high $\rightarrow$ underfitting.

10. INPUT NORMALIZATION

Technique	Formula	Result	Use Case
Min-Max Scaling	$x' = (x - \min) / (\max - \min)$	Values in $[0, 1]$	Known bounds
Standardization	$x' = (x - \mu) / \sigma$	Mean=0, Std=1	General purpose

► Why Normalize?

- **Faster convergence**
- Prevents one feature from dominating gradient updates
- Helps gradient descent find optimal path more efficiently

► Worked Example – Standardization

Data: $[2, 4, 6, 8]$ $\mu = 5$, $\sigma = 2.24 \rightarrow$ Normalized: $[-1.34, -0.45, 0.45, 1.34]$

11. BATCH NORMALIZATION

Concept

- **Normalizes the inputs to each layer within a mini-batch**
- Applied between linear transformation and activation function

$$\mu_B = (1/m) \cdot \sum x_i \quad \sigma^2_B = (1/m) \cdot \sum (x_i - \mu_B)^2 \quad \hat{x}_i = (x_i - \mu_B) / \sqrt{(\sigma^2_B + \epsilon)} \quad y = \gamma \cdot \hat{x}_i + \beta$$

Symbol	Meaning
μ_B	Mean of the mini-batch
σ^2_B	Variance of the mini-batch
ϵ	Small constant for numerical stability
γ, β	Learnable scale and shift parameters

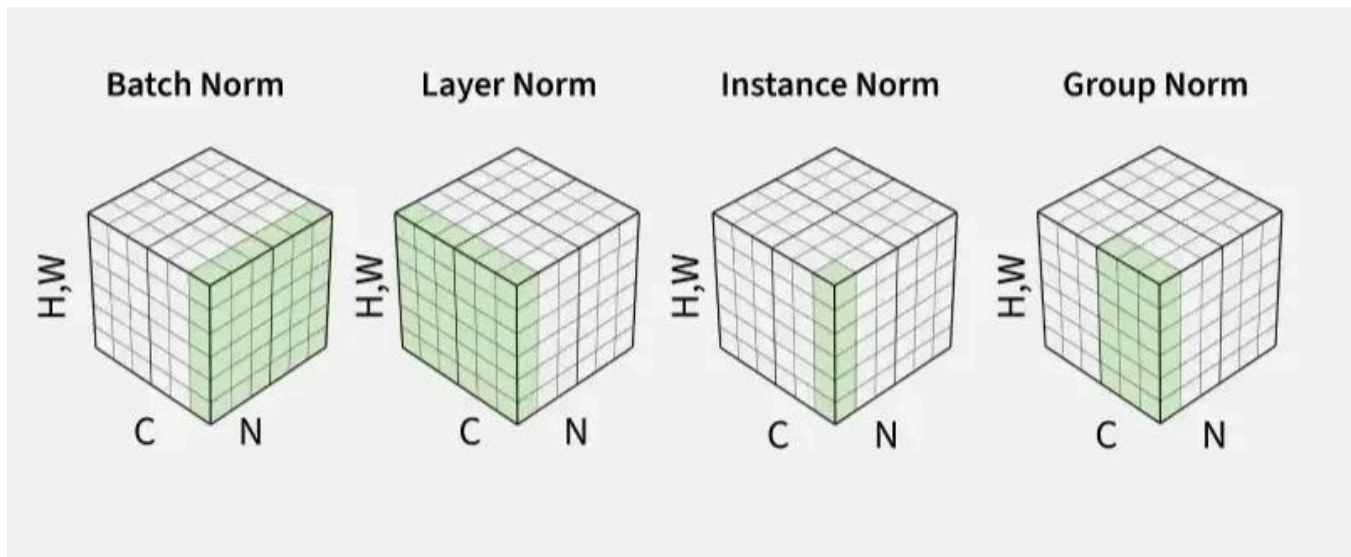
► Benefits

- **Faster training — allows higher learning rates**
- Reduces internal covariate shift
- Acts as mild regularization (reduces need for dropout)

12. LAYER NORMALIZATION

KEY DIFFERENCE: • Batch Normalization → normalizes ACROSS the batch (per feature) • Layer Normalization → normalizes ACROSS features (per sample)

Property	Batch Normalization	Layer Normalization
Normalizes Over	Batch dimension	Feature dimension
Batch Size Dependency	Yes — needs large batches	No — works with batch=1
Best For	CNNs, image models	RNNs, Transformers, NLP
Training vs Test	Different statistics	Same statistics



IMPORTANT DIFFERENCES & COMPARISONS

|Batch GD vs SGD vs Mini-Batch

Criterion	Batch GD	SGD	Mini-Batch GD
Data per update	Full dataset	1 sample	Batch of k samples
Speed	Slow	Fast per update	Fast + efficient
Stability	Stable	Very noisy	Balanced
Memory	High	Low	Moderate
Common Use	Rarely	Sometimes	Most common ✓

QUICK REVISION TABLE

Technique	Category	Key Role	Important Parameter
Batch Gradient Descent	Optimization	Full dataset update	Learning rate η
SGD	Optimization	Per-sample update	Learning rate η
Mini-Batch GD	Optimization	Batch update (best)	Batch size
Momentum	Optimization	Smooth gradient updates	β (0.9 typical)

RMSprop	Optimization	Adaptive per-param LR	β_2
Adam	Optimization	Adaptive + Momentum ★	$\beta_1, \beta_2, \epsilon$
L1 Regularization	Regularization	Sparse weights	λ (lambda)
L2 Regularization	Regularization	Weight decay	λ (lambda)
Dropout	Regularization	Random neuron drop	p (dropout rate)
Batch Normalization	Stabilization	Normalize per batch	γ, β (learnable)
Layer Normalization	Stabilization	Normalize per sample	γ, β (learnable)
Input Normalization	Preprocessing	Scale input features	μ, σ

HIGH PRIORITY EXAM TOPICS

★ Gradient descent update rule (know the formula $\theta = \theta - \eta \cdot \nabla J(\theta)$) ★ Adam optimizer (steps, formulas, default values) ★ Bias-variance trade-off (identify from behavior) ★ Overfitting vs underfitting (graph-based questions) ★ L1 vs L2 regularization (differences, sparse vs smooth) ★ Batch normalization (formula, benefits, symbols)

COMMON MISTAKES TO AVOID

- Using too-high learning rate → weight divergence (values blow up)
- Confusing bias with variance
- Ignoring normalization → slow or unstable training
- Using test set for tuning → data leakage, optimistic results
- Misinterpreting training vs validation error

KEY TERMS GLOSSARY

Learning Rate (η): Step size for weight updates in gradient descent

Epoch: One complete pass through the entire training dataset

Iteration: One weight update (one mini-batch processed)

Gradient: Partial derivative of loss w.r.t. parameters

Loss Function: Measures how wrong the model's predictions are

λ (Lambda): Regularization strength parameter

Momentum (β): Controls how much past gradient influences current update

Normalization: Scaling inputs/activations for stable training

Covariate Shift: Change in distribution of inputs to a layer during training

Sparse Weights: Many weights are zero (feature selection effect of L1)

END OF NOTES • Good Luck! 🎓